

# DE ZÉRO À L'EXCELLENCE : LE MANUEL PRATIQUE DJANGO & API (Application Programming Interface - Interface de Programmation d'Application) BACKEND

## Parcours d'Apprentissage Actif Pas-à-Pas : Du Premier Script à l'Architecture d'Entreprise Multi-Tenant

Ce manuel d'ingénierie prend un développeur débutant n'ayant aucune notion de Django et le guide par la pratique délibérée, les analogies visuelles et les défis progressifs jusqu'à la maîtrise des architectures SaaS (Software as a Service - Logiciel en tant que Service) critiques en production.

|                         |                                                                                                                           |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Stack Technique :       | Python 3.12+ • Django 5.x • DRF (Django REST Framework - Cadre d'Applications REST pour Django) • PostgreSQL Multi-tenant |
| Projet Fil Rouge :      | BuildTrack : Gestionnaire d'Équipements, Tâches et Incidents de Chantier BTP (Bâtiment et Travaux Publics)                |
| Piliers Scientifiques : | How Learning Works (2023) • Deep Reasoning Engine (Kahneman, Minto, Pólya, Meadows)                                       |
| Contrat Pédagogique :   | Règle des 70/30 (Pratique Active) • Zéro Prérequis Django • Zéro Rupture Cognitive                                        |
| Date & Version :        | Édition Intégrale 2.5 — 20 Pages d'Ingénierie Structurée Pas-à-Pas                                                        |

### LOOKING BACK & MÉTACOGNITION (George Pólya) : LE MANIFESTE DE L'APPRENTISSAGE PAR LA PRATIQUE

**Pourquoi ce livre va transformer votre cerveau** : La majorité des tutoriels en ligne vous font recopier du code sans vous faire réfléchir. Vous avez l'impression de comprendre (*Illusion de fluidité*), mais dès que vous vous retrouvez seul devant votre terminal, c'est le trou noir.

Ce manuel applique les **sciences cognitives** : vous allez manipuler des analogies concrètes, résoudre des énigmes, comprendre le 'Pourquoi' avant le 'Comment', tester des cas de crash réels (*Pre-Mortem*) et construire de vos propres mains un système complet dont vous comprendrez chaque engrenage.

**Règle de Transparence Linguistique** : Dès qu'un sigle technique apparaît pour la première fois dans cet ouvrage, sa signification complète en français et/ou anglais est systématiquement explicitée entre parenthèses pour ancrer vos modèles mentaux.

## PARTIE I : LES FONDATIONS PRATIQUES (DE ZÉRO)

### CHAPITRE 1 : Le Modèle Mental du Web & l'Écosystème Django

#### MODÈLE MENTAL & ANALOGIE CONCRÈTE : LE RESTAURANT DU WEB (CLIENT, ROUTEUR, VUE, ORM)

Imaginez un **restaurant gastronomique** :

1. **Le Client à table** (Le Navigateur Web / L'App Mobile) : Il ne va pas en cuisine. Il consulte le menu et émet une requête via le protocole HTTP (HyperText Transfer Protocol - Protocole de Transfert Hypertexte) : « *Apportez-moi la liste des chantiers en cours* ».
2. **Le Maître d'Hôtel (Le Routeur urls.py)** : Il réceptionne la commande et l'oriente vers le bon poste de travail en cuisine grâce à l'URL (Uniform Resource Locator - Localisateur Uniforme de Ressource).
3. **Le Chef de Cuisine (La Vue views.py)** : C'est le cerveau ! Il reçoit la commande, prépare le plat, va chercher les ingrédients et dresse l'assiette finale au format JSON (JavaScript Object Notation - Notation d'Objets JavaScript) ou HTML (HyperText Markup Language - Langage de Balisage Hypertexte).
4. **Le Garde-Manger (La Base de Données)** : Les données y sont entreposées.
5. **L'Économe (L'ORM / models.py)** : L'ORM (Object-Relational Mapping - Mappage Objet-Relationnel) sait exactement sur quelle étagère se trouve chaque donnée et la fournit au Chef sans que celui-ci ait besoin de fouiller dans la cave.

#### L'Architecture MTV (Model-Template-View - Modèle-Template-Vue)

Django repose sur le patron architectural **MTV**, une déclinaison pragmatique du classique **MVC** (Model-View-Controller - Modèle-Vue-Contrôleur). Le *Modèle* gère la persistance, le *Template* produit l'affichage visuel, et la *Vue* assure l'orchestration métier. Pour communiquer avec les serveurs de production, Django utilise les interfaces standardisées **WSGI** (Web Server Gateway Interface - Interface Passerelle de Serveur Web) pour le mode synchrone, ou **ASGI** (Asynchronous Server Gateway Interface - Interface Passerelle Asynchrone de Serveur) pour le mode temps réel asynchrone.

#### PIÈGE DÉBUTANT & SYSTÈME 1 (Kahneman) : MÉMORISATION SYNTAXIQUE VS MODÈLE MENTAL PROFOND

**L'Illusion du 'Je sais par cœur'** : Ne cherchez jamais à mémoriser chaque paramètre de Django. Les meilleurs architectes logiciels conservent la documentation officielle ouverte. Ce qui fait votre valeur, c'est votre **modèle mental** : savoir exactement à quelle étape du voyage HTTP une erreur s'est produite.

#### DÉFI STIMULANT EN ZONE PROXIMALE (Ericsson) : DÉFI STIMULANT #1 : DESSINEZ LE FLUX

##### Défi Actif #1 : Le Parcours de la Requête

Prenez une feuille et dessinez de mémoire le parcours d'une requête tapée dans le terminal ou le CLI (Command Line Interface - Interface en Ligne de Commande) :

Navigateur → HTTP GET /chantiers/ → urls.py → views.py → models.py → Base de données → Réponse JSON → Écran.

## CHAPITRE 2 : Votre Premier Projet Pas-à-Pas (Le Hello World Actif)

### 1. Initialisation de l'Environnement Virtuel & Installation

Ouvrez votre terminal dans votre IDE (Integrated Development Environment - Environnement de Développement Intégré) et tapez ces commandes fondamentales :

#### Fichier : Terminal (Bash/PowerShell)

```
# 1. Créer et activer un environnement virtuel isolé
python -m venv .venv
# Sur Windows : .venv\Scripts\activate | Sur Mac/Linux : source .venv/bin/activate

# 2. Installer Django (version 5+)
pip install django

# 3. Créer le squelette de configuration du projet
django-admin startproject config .

# 4. Créer notre première application métier
python manage.py startapp chantiers
```

### 2. Activer l'Application dans la Configuration Globale

Django ignore totalement une application tant qu'elle n'est pas déclarée. Ouvrez config/settings.py :

#### Fichier : config/settings.py

```
# config/settings.py
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Notre application métier BTP :
    'chantiers',
]
```

### 3. Votre Première Vue HTTP Réelle

#### Fichier : chantiers/views.py

```
# chantiers/views.py
from django.http import HttpResponse

def hello_chantier(request):
    return HttpResponse("<h1>Bienvenue sur BuildTrack : Le Cockpit BTP de SOUMAFE SARL !</h1>")
```

#### Fichier : config/urls.py

```
# config/urls.py
from django.contrib import admin
from django.urls import path
from chantiers.views import hello_chantier

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', hello_chantier, name='accueil'),
]
```

### DÉFI STIMULANT EN ZONE PROXIMALE (Ericsson) : DÉFI STIMULANT #2 : VOTRE PREMIER SERVEUR EN DIRECT

#### Défi Stimulant #2 : Première Victoire Active

1. Tapez python manage.py runserver dans votre terminal.
2. Ouvrez votre navigateur sur http://127.0.0.1:8000/.
3. Vous devez voir s'afficher votre message en grand. Vous venez de boucler votre premier cycle HTTP réel !

## CHAPITRE 3 : L'ORM Django : Vos Données sans Écrire de SQL

### MODÈLE MENTAL & ANALOGIE CONCRÈTE : LE TRADUCTEUR UNIVERSEL PYTHON <-> SQL

#### Pourquoi l'ORM est magique :

Sans ORM, pour créer un chantier en base de données, vous devriez écrire du SQL (Structured Query Language - Langage de Requêtes Structurées) brut : `INSERT INTO chantiers_projet (nom, ville) VALUES ('Tour Alpha', 'Abidjan');`.

Si demain votre entreprise bascule de SQLite à PostgreSQL ou Oracle, la syntaxe SQL peut changer.

L'ORM (Object-Relational Mapping - Mappage Objet-Relationnel) traduit automatiquement vos objets Python purs en tables dans votre SGBD (Système de Gestion de Base de Données) sans que vous ayez à rédiger une seule ligne de SQL !

### 1. Définition du Premier Modèle Métier : Projet de Chantier

#### Fichier : chantiers/models.py

```
# chantiers/models.py
from django.db import models

class Projet(models.Model):
    nom = models.CharField(max_length=150, verbose_name="Nom du Chantier")
    ville = models.CharField(max_length=100, default="Abidjan")
    budget_estime = models.DecimalField(max_digits=12, decimal_places=2, default=0)
    est_actif = models.BooleanField(default=True)
    cree_le = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.nom} ({self.ville})"
```

### 2. Le Rituel des Migrations en Deux Temps

Modifier models.py ne change RIEN à la base tant que vous n'avez pas exécuté le rituel sacré des migrations :

#### Fichier : Terminal (Le Rituel Sacré)

```
# Étape 1 : Django analyse models.py et prépare le plan de construction (le blueprint)
python manage.py makemigrations
# Sortie : Migrations for 'chantiers': 0001_initial.py - Create model Projet

# Étape 2 : Django exécute les commandes SQL réelles sur la base de données
python manage.py migrate
# Sortie : Applying chantiers.0001_initial... OK
```

### PIÈGE DÉBUTANT & SYSTÈME 1 (Kahneman) : LE PIÈGE N°1 DES DÉBUTANTS : L'OUBLI DE MIGRATE

**L'Angoisse du Fichier Fantôme** : Si vous ajoutez un champ dans models.py et lancez votre code sans faire makemigrations puis migrate, Django lèvera l'erreur redoutée : `OperationalError: no such column`. Retenez bien : **Code Modèle ≠ Base Réelle** tant que migrate n'a pas tourné.

## CHAPITRE 4 : Le Django Admin : Le Super-Pouvoir Instantané

### MODÈLE MENTAL & ANALOGIE CONCRÈTE : L'INTERFACE DE CONTRÔLE GÉNÉRÉE PAR INTROSPECTION

#### Le Cockpit Immédiat de l'Entreprise :

Dans d'autres frameworks (Node.js, Express, Spring Boot), concevoir une UI (User Interface - Interface Utilisateur) d'administration pour créer, modifier et filtrer les données prend plusieurs jours de travail frontend et backend.

Dans Django, cela prend exactement 30 secondes. Django lit la structure de vos modèles et génère automatiquement une interface Web sécurisée, responsive et ergonomique.

### 1. Créer son Compte Administrateur (Superuser)

Dans votre terminal, tapez la commande suivante et suivez les instructions interactives :

#### Fichier : Terminal (Création du Super-Utilisateur)

```
python manage.py createsuperuser
# Exemple de saisie :
# Nom d'utilisateur (leave blank to use 'admin'): admin
# Adresse électronique: admin@soumafe.ci
# Password: *****
# Superuser created successfully.
```

### 2. Enregistrer le Modèle dans l'Interface d'Admin

#### Fichier : chantiers/admin.py

```
# chantiers/admin.py
from django.contrib import admin
from chantiers.models import Projet

@admin.register(Projet)
class ProjetAdmin(admin.ModelAdmin):
    # Les colonnes visibles dans le tableau de bord
    list_display = ('nom', 'ville', 'budget_estime', 'est_actif', 'cree_le')

    # Filtres latéraux interactifs
    list_filter = ('est_actif', 'ville')

    # Barre de recherche plein texte sur le nom
    search_fields = ('nom',)

    # Trier par défaut par date de création décroissante
    ordering = ('-cree_le',)
```

### DÉFI STIMULANT EN ZONE PROXIMALE (Ericsson) : DÉFI STIMULANT #3 : CRÉATION DE DONNÉES EN DIRECT

#### Défi Stimulant #3 : Entrez dans le Cockpit !

1. Lancez `python manage.py runserver`.
2. Rendez-vous sur `http://127.0.0.1:8000/admin/` et connectez-vous.
3. Cliquez sur **Projets** puis **Ajouter Projet**. Créez 2 chantiers réels (ex: 'Immeuble Riviera' et 'Pont de Cocody').
4. Utilisez la barre de recherche et les filtres latéraux pour voir vos données réagir instantanément !

## CHAPITRE 5 : Manipuler les Données : Le Shell Interactif

### 1. Le Laboratoire Interactif & le CRUD Complet en Direct

Le Shell interactif de Django charge votre application et votre base en mémoire. Lancez : `python manage.py shell`. Vous y exécutez le **CRUD** (Create, Read, Update, Delete - Créer, Lire, Mettre à jour, Supprimer) :

#### Fichier : Python Shell (Le CRUD en Direct)

```
# Dans le shell interactif Python :
from chantiers.models import Projet

# 1. C (CREATE) : Créer un nouveau chantier
p = Projet.objects.create(nom="Tour Alpha", ville="Yamoussoukro", budget_estime=75000000)
print(p.id) # Une PK (Primary Key - Clé Primaire) auto-incrémentée a été attribuée

# 2. R (READ) : Interroger la base de données
tous = Projet.objects.all()
print("Nombre de projets :", tous.count())
abidjan = Projet.objects.filter(ville="Abidjan")
alpha = Projet.objects.get(nom__icontains="Alpha")

# 3. U (UPDATE) : Modifier un champ et sauvegarder
alpha.budget_estime = 80000000
alpha.save()

# 4. D (DELETE) : Supprimer l'objet
alpha.delete()
```

### 2. Lier les Tables : La Clé Étrangère ou FK (Foreign Key)

Sur un chantier, il y a des Tâches. Chaque tâche est rattachée à un seul Projet, mais un projet contient plusieurs tâches : c'est une **relation 1-N (Un à Plusieurs)** matérialisée par une **FK** :

#### Fichier : chantiers/models.py (Relation 1-N)

```
# chantiers/models.py (Ajout du modèle Tache)
class Tache(models.Model):
    # La clé étrangère qui relie la tâche à son projet parent
    projet = models.ForeignKey(Projet, on_delete=models.CASCADE, related_name="taches")
    titre = models.CharField(max_length=200)
    est_terminee = models.BooleanField(default=False)
    date_limite = models.DateField(null=True, blank=True)

    def __str__(self):
        return f"{self.titre} ({'Fait' if self.est_terminee else 'En attente'})"
```

### DÉFI STIMULANT EN ZONE PROXIMALE (Ericsson) : DÉFI STIMULANT #4 : EXPLORATION RELATIONNELLE

#### Défi Stimulant #4 : Navigation Inverse entre Objets !

Après avoir fait `makemigrations` et `migrate`, ouvrez le shell :

1. Créez un projet `p = Projet.objects.first()`.
2. Créez deux tâches : `Tache.objects.create(projet=p, titre='Coulage béton fondation')`.
3. Tapez : `p.taches.all()`. Observez comment Django remonte automatiquement toutes les tâches liées au projet !

## CHAPITRE 6 : Entrer dans l'Ère des API : DRF

### MODÈLE MENTAL & ANALOGIE CONCRÈTE : POURQUOI LES API JSON DOMINENT L'INDUSTRIE

Du Monolithe HTML aux API REST Découplées :

- **Web Ancien** : Le serveur générait tout le code HTML et l'envoyait au navigateur.
- **Web Moderne** : Le backend fournit uniquement des **données brutes en JSON** via une API REST (Representational State Transfer - Transfert d'État Représentationnel). Une application Next.js/React, une application mobile iOS/Android sur le chantier ou un script d'analyse consomment exactement les mêmes données. Votre backend devient un moteur universel.

### 1. Installer DRF & Comprendre le Rôle du Sérialiseur

Installez la bibliothèque : `pip install djangorestframework` puis ajoutez 'rest\_framework' dans vos `INSTALLED_APPS`. Le **Sérialiseur** prend les objets Python complexes de l'ORM et les convertit en JSON, et valide les JSON envoyés par le client.

Fichier : `chantiers/serializers.py`

```
# chantiers/serializers.py
from rest_framework import serializers
from chantiers.models import Projet

class ProjetSerializer(serializers.ModelSerializer):
    total_taches = serializers.IntegerField(source='taches.count', read_only=True)

    class Meta:
        model = Projet
        fields = ['id', 'nom', 'ville', 'budget_estime', 'est_actif', 'total_taches', 'cree_le']
```

### 2. Votre Première Vue API REST (APIView)

Fichier : `chantiers/views.py` (APIView REST)

```
# chantiers/views.py (Ajout des vues API REST)
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from chantiers.models import Projet
from chantiers.serializers import ProjetSerializer

class ProjetListCreateAPIView(APIView):
    def get(self, request):
        projets = Projet.objects.all().order_by('-cree_le')
        serializer = ProjetSerializer(projets, many=True)
        return Response(serializer.data, status=status.HTTP_200_OK)

    def post(self, request):
        serializer = ProjetSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

### DÉFI STIMULANT EN ZONE PROXIMALE (Ericsson) : DÉFI STIMULANT #5 : LE TEST DU NAVIGATEUR

**Défi Stimulant #5 : Connectez l'URL et Admirez la Browsable API**

Ajoutez `path('api/projets/', ProjetListCreateAPIView.as_view())` dans votre fichier `config/urls.py`.

Ouvrez votre navigateur sur `http://127.0.0.1:8000/api/projets/`. DRF vous affiche une console interactive où vous pouvez lire le JSON et soumettre des formulaires POST directement !

## CHAPITRE 7 : Validation Chirurgicale & Codes HTTP

### MODÈLE MENTAL & ANALOGIE CONCRÈTE : LES CODES HTTP : LA GRAMMAIRE UNIVERSELLE DU WEB

#### Les Codes HTTP : La Langue Internationale du Web :

Votre serveur ne doit JAMAIS renvoyer un statut 200 OK quand une erreur s'est produite.

- **200 OK** : La lecture s'est déroulée avec succès.
- **201 Created** : La nouvelle ressource a été créée en base avec succès.
- **400 Bad Request** : Données envoyées invalides ou incomplètes (rejetées par la validation).
- **401 Unauthorized** : Vous devez être authentifié pour accéder à cette ressource.
- **403 Forbidden** : Vous êtes authentifié, mais vous n'avez pas les droits (rôle insuffisant).
- **404 Not Found** : La ressource demandée n'existe pas.

### Validation Métier dans le Sérialiseur

Vous disposez de deux niveaux de validation : champ par champ ou validation globale multi-champs :

#### Fichier : chantiers/serializers.py (Règles Métier)

```
# chantiers/serializers.py (Validation Avancée)
class ProjetSerializer(serializers.ModelSerializer):
    class Meta:
        model = Projet
        fields = ['id', 'nom', 'ville', 'budget_estime', 'est_actif', 'cree_le']

# Niveau 1 : Validation ciblée d'un champ unique (format: validate_<nom_du_champ>)
def validate_budget_estime(self, value):
    if value < 0:
        raise serializers.ValidationError("Le budget estimé d'un chantier ne peut pas être négatif.")
    if value > 500000000000:
        raise serializers.ValidationError("Montant exorbitant : vérifiez la devise (plafond à 50 Mds FCFA).")
    return value

# Niveau 2 : Validation croisée de plusieurs champs
def validate(self, attrs):
    nom = attrs.get('nom', '')
    ville = attrs.get('ville', '')
    if 'abidjan' in ville.lower() and len(nom) < 5:
        raise serializers.ValidationError({"nom": "Pour la zone d'Abidjan, le libellé du chantier doit être précis (min 5 car)."})
    return attrs
```

### DÉFI STIMULANT EN ZONE PROXIMALE (Ericsson) : DÉFI STIMULANT #6 : LE TEST DU CHAMP NÉGATIF

#### Défi Stimulant #6 : Le Crash Test des Données Pourries

Ouvrez la Browsable API ou votre terminal :

Tentez d'envoyer un POST avec : {"nom": "Test", "budget\_estime": -5000}.

Constater que DRF répond immédiatement un code **400 Bad Request** avec le message d'erreur précis au format JSON. Votre base de données est protégée contre la corruption !

## CHAPITRE 8 : Authentification & Sécurisation des Endpoints

### MODÈLE MENTAL & ANALOGIE CONCRÈTE : LA DISTINCTION VITALE : IDENTITÉ VS POUVOIRS

Authentification vs Autorisation (RBAC) :

- **Authentification (Qui êtes-vous ?)** : Prouver son identité via un login/mot de passe ou un jeton JWT (JSON Web Token - Jeton Web JSON).
- **Autorisation (Qu'avez-vous le droit de faire ?)** : Le **RBAC** (Role-Based Access Control - Contrôle d'Accès Basé sur les Rôles) définit les permissions. Un ouvrier peut consulter le planning, mais seul le Conducteur de Travaux peut valider un bon de commande.

### Créer une Permission DRF Sur-Mesure

Par défaut, DRF propose `IsAuthenticated` (accessible à tout connecté) et `IsAdminUser`. En entreprise, on conçoit des permissions chirurgicales :

Fichier : `chantiers/permissions.py`

```
# chantiers/permissions.py
from rest_framework import permissions

class EstChefDeChantierOuLectureSeule(permissions.BasePermission):
    """Lecture libre pour les connectés, modification réservée aux staffs/admins."""
    def has_permission(self, request, view):
        # Les méthodes SAFE_METHODS sont : GET, HEAD, OPTIONS (lecture seule)
        if request.method in permissions.SAFE_METHODS:
            return True
        # Pour POST, PUT, DELETE : l'utilisateur doit être membre du staff
        return bool(request.user and request.user.is_staff)
```

Fichier : `chantiers/views.py` (Sécurisation)

```
# chantiers/views.py (Protection des Endpoints)
from rest_framework.permissions import IsAuthenticated
from chantiers.permissions import EstChefDeChantierOuLectureSeule

class ProjetListCreateAPIView(APIView):
    # Les deux gardiens à l'entrée :
    permission_classes = [IsAuthenticated, EstChefDeChantierOuLectureSeule]

    # ... le reste du code des méthodes get() et post() ...
```

### DÉFI STIMULANT EN ZONE PROXIMALE (Ericsson) : DÉFI STIMULANT #7 : SIMULATION DE BRÈCHE

Défi Stimulant #7 : Le Test de l'Intrus

1. Appelez l'API en GET sans être connecté → Observez le code **401 UNAUTHORIZED**.
  2. Connectez-vous avec un utilisateur simple non-staff et tentez un POST → Observez le code **403 FORBIDDEN** !
- Votre API dispose désormais d'un bouclier de protection inviolable.

## PARTIE II : L'ÉLÉVATION VERS LE NIVEAU ENTREPRISE

### CHAPITRE 9 : Pourquoi l'Approche Débutant Explode en Entreprise

#### MODÈLE MENTAL & ANALOGIE CONCRÈTE : DE LA CABANE AU GRATTE-CIEL D'ENTREPRISE

**L'analogie de la Cabane vs Gratte-Ciel (Donella Meadows & Barbara Minto) :**

Pour construire une cabane en bois au fond du jardin, vous pouvez mettre tous vos outils par terre dans la même pièce. C'est ce que font les débutants en mettant toute leur logique dans `views.py` ou `models.py`.

Mais pour ériger une tour de 30 étages (la plateforme SaaS critique de SOUMAFE SARL), cette méthode entraîne l'effondrement immédiat du bâtiment : effets de bord invisibles, régressions massives, requêtes SQL qui font planter le serveur et impossibilité de tester unitairement.

#### Le Patron CQS (Command-Query Separation) & DDD (Domain-Driven Design)

Pour supporter des millions de requêtes sans faille, nous appliquons une séparation stricte entre **la Lecture pure (Selectors)** et **l'Écriture métier (Services)** :

| Couche Métier                      | Rôle Strict                                               | Ce qu'on y trouve                                                                              |
|------------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------|
| 1. Sélecteurs (selectors/)         | Lecture seule optimisée.                                  | Requêtes avec <code>select_related</code> , zéro écriture, retour <code>QuerySets</code> .     |
| 2. Services (services/)            | Écriture métier transactionnelle.                         | Fonctions décorées de <code>@transaction.atomic</code> , règles RG, <code>soft-delete</code> . |
| 3. Sérialiseurs DTO (serializers/) | Validation DTO (Data Transfer Object) d'entrée et sortie. | Validation des formats et types, transformation en JSON.                                       |
| 4. Permissions (permissions.py)    | Contrôle d'accès cumulatif.                               | Niveau 1 (Rôle Global) + Niveau 2 (Affectation au chantier).                                   |
| 5. Vues API (views/)               | Contrôleur HTTP mince.                                    | Dispatch des verbes HTTP, pagination, documentation OpenAPI Swagger.                           |

#### PRE-MORTEM : SIMULATION DU CRASH (Kahneman) : DÉCOUPLAGE TOTAL DU PROTOCOLE HTTP

**Simulation Pre-Mortem (Daniel Kahneman) :**

Pourquoi est-il formellement interdit d'écrire `from rest_framework.request import Request` dans un Service ?

**Conséquence mortelle :** Si votre logique métier dépend de l'objet request HTTP, vous ne pourrez JAMAIS appeler ce service depuis une tâche asynchrone Celery en tâche de fond, ni depuis un script de commande dans le terminal, ni depuis un test unitaire sans devoir simuler tout un serveur HTTP ! Les services doivent rester des **fonctions Python pures** indépendantes du web.

## CHAPITRE 10 : Restructuration Physique : De l'App à Plat aux Packages Modulaires

*Étape Charnière : Comprendre la Métamorphose des Fichiers et la Découverte par Django*

### MODÈLE MENTAL & ANALOGIE CONCRÈTE : TRANSFORMER DES FICHIERS EN PACKAGES PYTHON

#### De la Feuille Volante au Classeur Structuré :

Dans l'approche débutant (Partie I), tous les modèles étaient entassés dans un unique `models.py`, et toutes les vues dans `views.py`. Dans un système d'entreprise gérant 50 tables et 200 règles de gestion, ces fichiers atteignent 3000 lignes et deviennent illisibles.

**La règle d'or :** En Python, un dossier contenant un fichier `__init__.py` est traité exactement comme un fichier de code (un *Package*). Nous transformons donc nos fichiers uniques en répertoires spécialisés.

### L'Arborescence Complète d'une Application d'Entreprise Modularisée

#### Fichier : Structure Physique du Répertoire `apps/chantier/`

```
apps/chantier/
■■■ __init__.py
■■■ apps.py                <-- Configuration de l'application (name = 'apps.chantier')
■■■ models/
■   ■■■ __init__.py        <-- POINT CRITIQUE VITAL : Exporte les modèles pour makemigrations !
■   ■■■ projet.py          <-- Modèle métier Projet isolé
■■■ selectors/
■   ■■■ __init__.py        <-- Fonctions pures de LECTURE SQL optimisées
■   ■■■ projet.py          <-- Fonctions pures d'ÉCRITURE métier transactionnelles
■■■ services/
■   ■■■ __init__.py        <-- Fonctions pures d'ÉCRITURE métier transactionnelles
■   ■■■ projet.py          <-- Fonctions pures d'ÉCRITURE métier transactionnelles
■■■ serializers/
■   ■■■ __init__.py        <-- DTOs d'entrée et de sortie
■   ■■■ projet.py          <-- DTOs d'entrée et de sortie
■■■ views/
■   ■■■ __init__.py        <-- Contrôleurs HTTP minces
■   ■■■ projet.py          <-- Contrôleurs HTTP minces
```

### PIÈGE DÉBUTANT & SYSTÈME 1 (Kahneman) : LE SECRET DE LA DÉCOUVERTE DES MODÈLES PAR DJANGO

#### LE PIÈGE MORTEL DU `__init__.py` DANS `models/` :

Lorsque vous remplacez `models.py` par un dossier `models/`, Django cherche à importer `apps.chantier.models`.

Si votre fichier `apps/chantier/models/__init__.py` est VIDE, Django ne charge pas le sous-fichier `projet.py`. Conséquence catastrophique : lors du prochain `makemigrations`, Django pense que vous avez supprimé votre table et propose d'effacer toutes vos données !

**L'action corrective obligatoire :** Ouvrez `apps/chantier/models/__init__.py` et écrivez explicitement :

```
from .projet import Projet (et ainsi pour chaque modèle ajouté au projet).
```

### DÉFI STIMULANT EN ZONE PROXIMALE (Ericsson) : DÉFI STIMULANT #8 : VALIDATION DE LA STRUCTURE MODULAIRE

#### Défi Stimulant #8 : Déclaration dans `INSTALLED_APPS`

Dans `config/settings.py`, remplacez `'chantiers'` par `'apps.chantier'`.

Dans `apps/chantier/apps.py`, configurez `name = 'apps.chantier'`.

Lancez `python manage.py check` : le message magique doit s'afficher : « *System check identified no issues* » !

## CHAPITRE 11 : Atelier de Refactoring Guidé : La Métamorphose de Projet Pas-à-Pas

*Le Pont Cognitif : Prendre le Code Connu du Chapitre 6 et le Nettoyer en Direct*

### MODÈLE MENTAL & ANALOGIE CONCRÈTE : LE REFACTORING PAS-À-PAS DU MODÈLE PROJET

#### L'Opération Chirurgicale sur votre Propre Code :

Pour bien comprendre, nous n'allons pas inventer un nouveau problème. Nous reprenons exactement la vue `ProjetListCreateAPIView` créée au Chapitre 6, et nous allons extraire ses organes un par un :

1. La requête `Projet.objects.all()` déménage dans un **Sélecteur**.
2. L'écriture `serializer.save()` est bannie et remplacée par un appel à un **Service** transactionnel.
3. La vue s'amincit et devient un simple aiguilleur !

#### Fichier : `apps/chantier/selectors/projet.py` (Étape 1 : Extraction Sélecteur)

```
# apps/chantier/selectors/projet.py
from apps.chantier.models.projet import Projet

def projets_liste(*, ville=None, est_actif=True):
    """Lecture pure : retourne la liste filtrée sans aucun effet de bord."""
    qs = Projet.objects.filter(est_actif=est_actif)
    if ville:
        qs = qs.filter(ville__iexact=ville)
    return qs.order_by("-cree_le")
```

#### Fichier : `apps/chantier/services/projet.py` (Étape 2 : Extraction Service)

```
# apps/chantier/services/projet.py
from django.db import transaction
from apps.chantier.models.projet import Projet

@transaction.atomic
def creer_projet(*, nom, ville="Abidjan", budget_estime=0):
    """Écriture métier transactionnelle : garantit la cohérence en base."""
    projet = Projet(
        nom=nom.strip(),
        ville=ville.strip(),
        budget_estime=budget_estime,
        est_actif=True
    )
    projet.full_clean() # Exécute toutes les validations de modèles
    projet.save()
    return projet
```

#### Fichier : `apps/chantier/views/projet.py` (Résultat : Le Contrôleur Mince)

```
# apps/chantier/views/projet.py (Étape 3 : La Vue Allégée)
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from apps.chantier.selectors.projet import projets_liste
from apps.chantier.services.projet import creer_projet
from apps.chantier.serializers.projet import ProjetSerializer

class ProjetListCreateAPIView(APIView):
    def get(self, request):
        projets = projets_liste(ville=request.GET.get('ville')) # Délégation lecture
        return Response(ProjetSerializer(projets, many=True).data, status=status.HTTP_200_OK)

    def post(self, request):
        serializer = ProjetSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        # BANNISSEMENT DE serializer.save() -> Appel du Service d'entreprise :
        projet = creer_projet(**serializer.validated_data)
        return Response(ProjetSerializer(projet).data, status=status.HTTP_201_CREATED)
```

### LOOKING BACK & MÉTACOGNITION (George Pólya) : LA VICTOIRE DU DÉCOUPLAGE

**Regardez votre nouveau code :** La vue ne contient plus AUCUNE requête SQL et n'écrit plus directement en base. Si demain vous devez créer un chantier lors de l'import d'un fichier Excel en ligne de commande, vous appelez simplement `creer_projet(nom=...)` sans avoir besoin de simuler une fausse requête web !

## CHAPITRE 12 : Le Socle Commun (UUID, Soft-Delete & Sécurité SQL)

### Les Invariants Techniques Obligatoires de la Plateforme d'Entreprise

#### CADRAGE STRATÉGIQUE SCQA (Minto) : LES 4 LOIS DU SOCLE COMMUN D'ENTREPRISE

Les 4 Lois Sacrées du Socle d'Entreprise :

1. **Bannissement des ID Entiers (1, 2, 3...)** : Remplacés par des **UUID** (Universally Unique Identifier - Identifiant Universel Unique) v4. Pourquoi ? Un entier séquentiel permet à un concurrent de deviner vos volumes de ventes ou d'injecter des ID voisins dans l'URL. De plus, sur une application mobile hors-ligne, le technicien peut générer un UUID local sans risque de collision lors de la synchronisation !
2. **Horodatage UTC Obligatoire** : Toutes les dates sont stockées en **UTC** (Coordinated Universal Time - Temps Universel Coordonné) avec **TIMESTAMPZ**.
3. **La Suppression Physique n'Existe Pas** : Utilisation exclusive du **Soft-Delete** (marquage logique d'horodatage).
4. **Protection Vitale RESTRICT** : Interdiction formelle du `on_delete=CASCADE` sur les données sensibles.

#### Fichier : `apps/core/models/__init__.py` (Le Socle Commun)

```
# apps/core/models/__init__.py (Le Socle Commun Abstrait)
import uuid
from django.db import models
from django.utils import timezone

class ManagerActif(models.Manager):
    """Injecte automatiquement WHERE supprime_le IS NULL sur chaque requête SQL."""
    def get_queryset(self):
        return super().get_queryset().filter(supprime_le__isnull=True)

class ManagerCompleter(models.Manager):
    """Permet aux auditeurs et administrateurs d'accéder aux données archivées."""
    pass

class ModeleBase(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    cree_le = models.DateTimeField(auto_now_add=True)
    modifie_le = models.DateTimeField(auto_now=True)
    cree_par = models.ForeignKey('accounts.Utilisateur', on_delete=models.RESTRICT, null=True, blank=True, related_name='+')

    # Soft-delete : non nul = ligne supprimée logiquement
    supprime_le = models.DateTimeField(null=True, blank=True, db_index=True)
    supprime_par = models.ForeignKey('accounts.Utilisateur', on_delete=models.RESTRICT, null=True, blank=True, related_name='+')

    objects = ManagerActif() # Manager par défaut : filtre les supprimés
    tous_objets = ManagerCompleter() # Manager pour restauration et audit légal

    class Meta:
        abstract = True # Modèle abstrait : ne crée pas de table SQL 'ModeleBase'

    def delete(self, using=None, keep_parents=False, utilisateur=None):
        """Détourne la suppression destructive vers un archivage logique horodaté."""
        self.supprime_le = timezone.now()
        self.supprime_par = utilisateur
        self.save(update_fields=['supprime_le', 'supprime_par', 'modifie_le'])
```

#### PIÈGE DÉBUTANT & SYSTÈME 1 (Kahneman) : LE DANGER MORTEL DE CASCADE EN PRODUCTION

##### POURQUOI `on_delete=models.RESTRICT` ET JAMAIS CASCADE ?

Dans les tutoriels pour débutants, on met toujours `CASCADE`. En entreprise, si un gestionnaire supprime un Projet par mégarde, `CASCADE` efface silencieusement en cascade 500 tâches, 200 incidents, 80 devis et toutes les factures associées !

Avec `RESTRICT`, la base de données SQL lève une erreur et **interdit la suppression** tant qu'il subsiste des enregistrements liés. Vos données d'entreprise sont sanctuarisées.

## CHAPITRE 13 : Le Découplage des Sérialiseurs DTO & Le Double Verrou

### Dissocier l'Entrée de la Sortie et Sécuriser aux Niveaux Global et Objet

#### MODÈLE MENTAL & ANALOGIE CONCRÈTE : DTO D'ENTRÉE VS DTO DE SORTIE

##### Pourquoi un seul Sérialiseur ne suffit plus en Entreprise :

En lecture (GET), le client veut un JSON riche avec les noms complets des chantiers et des utilisateurs.

En écriture (POST), le client n'envoie que des identifiants et des valeurs brutes.

Vouloir faire les deux avec le même ModelSerializer crée des failles de sécurité et du code spaghetti. Nous créons deux **DTO** (Data Transfer Object - Objet de Transfert de Données) distincts : un pour l'entrée, un pour la sortie.

#### Fichier : Exemple de Découplage DTO In / Out

```
# Anatomie chirurgicale des deux DTOs :
# 1. DTO de Sortie : ModelSerializer pour formater la réponse JSON
class ProjetListSerializer(serializers.ModelSerializer):
    chef_nom = serializers.CharField(source="chef.nom_complet", read_only=True)
    class Meta:
        model = Projet
        fields = ["id", "nom", "ville", "budget_estime", "chef_nom"]

# 2. DTO d'Entrée : Serializer pur pour valider les données brutes
class ProjetCreateSerializer(serializers.Serializer):
    nom = serializers.CharField(max_length=150)
    ville = serializers.CharField(max_length=100, default="Abidjan")
    budget_estime = serializers.DecimalField(max_digits=12, decimal_places=2)
    # L'argument magique source="chef" charge l'instance Utilisateur en mémoire :
    chef_id = serializers.PrimaryKeyRelatedField(queryset=Utilisateur.objects.all(), source="chef")
```

#### La Muraille de Sécurité RBAC à Double Verrou

En entreprise, vérifier que l'utilisateur est connecté ne suffit pas. Nous imposons deux verrous hermétiques :

#### Fichier : apps/chantier/permissions.py (Le Double Verrou)

```
# apps/chantier/permissions.py
from rest_framework import permissions
from apps.core.permissions import RoleRequis, MembreDuProjet

class PeutGererProjet(permissions.BasePermission):
    # VERROU NIVEAU 1 : Rôle global requis au sein de l'entreprise
    def has_permission(self, request, view):
        return RoleRequis.pour('DIRECTEUR', 'CONDUCTEUR_TRAVAUX')().has_permission(request, view)

    # VERROU NIVEAU 2 : L'utilisateur est-il affecté à CE chantier spécifique ?
    def has_object_permission(self, request, view, obj):
        return MembreDuProjet().has_object_permission(request, view, obj)
```

#### PIÈGE DÉBUTANT & SYSTÈME 1 (Kahneman) : LE PIÈGE DU CHECK\_OBJECT\_PERMISSIONS NON AUTOMATISÉ

##### LE PIÈGE INVISIBLE DE L'APIView :

DRF exécute automatiquement `has_permission` sur toutes les vues.

Cependant, dans une APIView personnalisée, DRF **n'exécute JAMAIS automatiquement** `has_object_permission` ! Si vous oubliez d'appeler explicitement `self.check_object_permissions(request, obj)` dans votre méthode de vue, le second verrou ne s'enclenche pas ! C'est pourquoi nous configurons également `get_permissions(self)` dynamiquement.

## CHAPITRE 14 : Le Cas Réel d'Entreprise : API Incidents de Chantier

### Étape 1 : Modélisation Métier Robuste & Enums TextChoices

#### CADRAGE STRATÉGIQUE SCQA (Minto) : GESTION ET TRAÇABILITÉ DES INCIDENTS

**Situation** : Sur les chantiers de BTP gérés par SOUMAFE SARL, des incidents surviennent (intempéries, pannes d'engins, accidents du travail).

**Complication** : Si un incident critique n'est pas documenté et résolu sous 24h, le chantier s'arrête avec des pénalités financières colossales.

**Question** : Comment concevoir un endpoint hautement sécurisé pour déclarer et résoudre des incidents avec traçabilité intégrale ?

**Answer** : Une architecture CQS complète avec soft-delete, transactions atomiques et documentation OpenAPI.

#### Modélisation avec Énumérations Strictes (TextChoices)

Fichier : apps/chantier/models/incident.py

```
# apps/chantier/models/incident.py
from django.db import models
from django.utils import timezone
from django.utils.translation import gettext_lazy as _
from apps.core.models import ModeleBase

class GraviteIncident(models.TextChoices):
    FAIBLE = "FAIBLE", _("Faible - Sans impact immédiat")
    MOYENNE = "MOYENNE", _("Moyenne - Ralentissement des travaux")
    CRITIQUE = "CRITIQUE", _("Critique - Arrêt immédiat / Danger corporel")

class StatutIncident(models.TextChoices):
    DECLARE = "DECLARE", _("Déclaré sur le terrain")
    EN_TRAITEMENT = "EN_TRAITEMENT", _("En cours de traitement")
    RESOLU = "RESOLU", _("Résolu avec action corrective")
    CLOTURE = "CLOTURE", _("Clôturé par la direction")

class IncidentChantier(ModeleBase):
    projet = models.ForeignKey("projets.Projet", on_delete=models.RESTRICT, related_name="incidents")
    lot = models.ForeignKey("projets.Lot", on_delete=models.RESTRICT, null=True, blank=True, related_name="incidents")
    titre = models.CharField(max_length=200, verbose_name=_("Titre"))
    description = models.TextField(verbose_name=_("Description détaillée"))
    gravite = models.CharField(max_length=20, choices=GraviteIncident.choices, default=GraviteIncident.MOYENNE, db_index=True)
    statut = models.CharField(max_length=20, choices=StatutIncident.choices, default=StatutIncident.DECLARE, db_index=True)
    date_survenance = models.DateTimeField(default=timezone.now)
    action_corrective = models.TextField(blank=True, default="", verbose_name=_("Action corrective apportée"))
    resolu_le = models.DateTimeField(null=True, blank=True)
    declare_par = models.ForeignKey("accounts.Utilisateur", on_delete=models.RESTRICT, related_name="incidents_declares")

    class Meta:
        verbose_name = _("Incident de chantier")
        ordering = ["-date_survenance"]

    def __str__(self):
        return f"[{self.gravite}] {self.titre} ({self.projet.nom})"
```

#### DÉFI STIMULANT EN ZONE PROXIMALE (Ericsson) : DÉFI STIMULANT #9 : LE MODÈLE EN PRODUCTION

##### Défi Stimulant #9 : Exportation & Migration Immédiate

1. N'oubliez pas d'ajouter from .incident import IncidentChantier, GraviteIncident, StatutIncident dans apps/chantier/models/\_\_init\_\_.py.

2. Lancez python manage.py makemigrations puis python manage.py migrate.

Observez comment PostgreSQL crée la table avec la clé primaire UUID et les contraintes RESTRICT !

## CHAPITRE 14 (Suite) : Sélecteurs Purs & Services Métier

### Étape 2 & 3 : Séparation Hermétique entre Requêtes SQL et Mutations Métier

#### 1. Le Sélecteur de Lecture Pure (Zéro Effet de Bord)

Fichier : `apps/chantier/selectors/incident.py`

```
# apps/chantier/selectors/incident.py
from apps.chantier.models.incident import IncidentChantier

def incidents_liste(*, projet_id=None, statut=None, gravite=None):
    # Élimination du problème N+1 via select_related pour joindre en une seule requête SQL :
    qs = IncidentChantier.objects.select_related("projet", "lot", "declare_par").all()
    if projet_id: qs = qs.filter(projet_id=projet_id)
    if statut:    qs = qs.filter(statut=statut)
    if gravite:   qs = qs.filter(gravite=gravite)
    return qs.order_by("-date_survenance")

def incident_par_id(incident_id):
    try:
        return IncidentChantier.objects.select_related("projet", "lot", "declare_par").get(id=incident_id)
    except (IncidentChantier.DoesNotExist, ValueError):
        return None
```

#### 2. Le Service d'Écriture Métier sous Transaction Atomique

Fichier : `apps/chantier/services/incident.py`

```
# apps/chantier/services/incident.py
from django.core.exceptions import ValidationError
from django.db import transaction
from django.utils import timezone
from apps.chantier.models.incident import IncidentChantier, GraviteIncident, StatutIncident

@transaction.atomic
def creer_incident(*, projet, titre, description, declare_par, gravite=GraviteIncident.MOYENNE, lot=None, date_survenance=None):
    # Règle de Gestion RG-02 : Vérifier que le lot appartient bien à ce projet précis
    if lot is not None and lot.projet_id != projet.id:
        raise ValidationError({"lot": "Le lot sélectionné n'appartient pas au chantier spécifié."})

    incident = IncidentChantier(
        projet=projet, lot=lot, titre=titre.strip(), description=description.strip(),
        gravite=gravite, statut=StatutIncident.DECLARE,
        date_survenance=date_survenance or timezone.now(),
        declare_par=declare_par, cree_par=declare_par
    )
    incident.full_clean() # Validation obligatoire des contraintes de modèle
    incident.save()
    return incident

@transaction.atomic
def resoudre_incident(*, incident, action_corrective, utilisateur):
    if not action_corrective.strip():
        raise ValidationError({"action_corrective": "Une action corrective documentée est obligatoire."})
    incident.statut = StatutIncident.RESOLU
    incident.action_corrective = action_corrective.strip()
    incident.resolu_le = timezone.now()
    # Optimisation SQL : update_fields n'écrit que les colonnes ciblées
    incident.save(update_fields=["statut", "action_corrective", "resolu_le", "modifie_le"])
    return incident
```

## CHAPITRE 14 (Suite) : Vues API REST, Double Verrou & Swagger UI

### Étape 4, 5 & 6 : Le Contrôleur Mince et l'Intégration dans le Routage Multi-Tenant

Fichier : `apps/chantier/serializers/incident.py`

```
# apps/chantier/serializers/incident.py (DTOs Entrée / Sortie)
from rest_framework import serializers
from apps.chantier.models.incident import IncidentChantier, GraviteIncident
from apps.projets.models import Lot, Projet

class IncidentListSerializer(serializers.ModelSerializer):
    projet_nom = serializers.CharField(source="projet.nom", read_only=True)
    declare_par_nom = serializers.CharField(source="declare_par.nom_complet", read_only=True)
    class Meta:
        model = IncidentChantier
        fields = ["id", "titre", "gravite", "statut", "date_survenance", "projet_nom", "declare_par_nom", "resolu_le"]

class IncidentCreateSerializer(serializers.Serializer):
    projet_id = serializers.PrimaryKeyRelatedField(queryset=Projet.objects.all(), source="projet")
    lot_id = serializers.PrimaryKeyRelatedField(queryset=Lot.objects.all(), source="lot", required=False, allow_null=True)
    titre = serializers.CharField(max_length=200)
    description = serializers.CharField()
    gravite = serializers.ChoiceField(choices=GraviteIncident.choices, default=GraviteIncident.MOYENNE)
    date_survenance = serializers.DateTimeField(required=False)
```

Fichier : `apps/chantier/views/incident.py`

```
# apps/chantier/views/incident.py
from drf_spectacular.utils import extend_schema
from rest_framework import status
from rest_framework.parsers import JSONParser
from rest_framework.permissions import IsAuthenticated
from rest_framework.response import Response
from rest_framework.views import APIView
from apps.chantier.selectors.incident import incidents_liste
from apps.chantier.serializers.incident import IncidentCreateSerializer, IncidentListSerializer
from apps.chantier.services.incident import creer_incident
from apps.core.pagination import PaginationStandard

class IncidentListCreateView(APIView):
    parser_classes = [JSONParser]
    pagination_class = PaginationStandard

    @extend_schema(summary="Lister les incidents de chantier", responses={200: IncidentListSerializer(many=True)})
    def get(self, request):
        qs = incidents_liste(projet_id=request.GET.get('projet'))
        paginator = self.pagination_class()
        page = paginator.paginate_queryset(qs, request)
        return paginator.get_paginated_response(IncidentListSerializer(page, many=True).data)

    @extend_schema(summary="Déclarer un nouvel incident", request=IncidentCreateSerializer, responses={201: IncidentListSerializer})
    def post(self, request):
        serializer = IncidentCreateSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        incident = creer_incident(declare_par=request.user, **serializer.validated_data)
        return Response(IncidentListSerializer(incident).data, status=status.HTTP_201_CREATED)
```

### CADRAGE STRATÉGIQUE SCQA (Minto) : LA TUYAUTERIE DU ROUTAGE MULTI-TENANT

**Routage Automatique Multi-Tenant dans CCD (Contrôle et Conduite Digitale) Digital :**

1. **Bifurcation des Plans d'URL** : `config/urls.py` gère l'accueil public, tandis que `config/urls_tenant.py` gère l'espace privé isolé de chaque entreprise cliente.
2. Déclarez la route dans `apps/chantier/urls.py` : `path('incidents/', IncidentListCreateView.as_view(), name='incident-liste-creer')`.
3. Dans `config/urls_tenant.py`, l'application est branchée sous `api/v1/` : `path('api/v1/', include('apps.chantier.urls'))`.
4. Votre endpoint est immédiatement accessible sur `/api/v1/incidents/` et auto-documenté dans Swagger UI OpenAPI (Open Application Programming Interface) !

## CHAPITRE 15 : Tests Automatisés (Pytest) & Définition of Done (DoD)

### Phase 4 : TDD (Test-Driven Development), Validation Formelle & Prévention des Régressions

#### MODÈLE MENTAL & ANALOGIE CONCRÈTE : LE CODE QUI SURVEILLE LE CODE

##### Le Filet de Sécurité du Trapéziste :

Développer sans tests automatisés, c'est exécuter un saut périlleux sans filet de sécurité à 20 mètres de hauteur.

En appliquant le **TDD** (Test-Driven Development - Développement Piloté par les Tests), chaque test unitaire devient un gardien infatigable qui vérifie 24h/24 qu'aucune régression n'a été introduite dans le code de production.

##### Fichier : apps/chantier/tests/test\_incidents\_api.py

```
# apps/chantier/tests/test_incidents_api.py
import pytest
from rest_framework import status
from apps.chantier.models.incident import GraviteIncident, IncidentChantier

@pytest.mark.django_db
class TestIncidentAPI:
    def test_creeer_incident_valide_succes(self, client_conducteur, chantier_abidjan):
        """Vérifie qu'un conducteur de travaux peut déclarer un incident valide."""
        url = "/api/v1/incidents/"
        payload = {
            "projet_id": str(chantier_abidjan.id),
            "titre": "Fissure constatée sur le voile béton B2",
            "description": "Arrêt préventif de la zone en attente de l'avis du bureau d'études.",
            "gravite": GraviteIncident.CRITIQUE
        }
        response = client_conducteur.post(url, data=payload, format="json")
        assert response.status_code == status.HTTP_201_CREATED
        assert IncidentChantier.objects.count() == 1
        incident = IncidentChantier.objects.first()
        assert incident.titre == payload["titre"]
        assert incident.supprime_le is None # Vérification du soft-delete actif

    def test_interdiction_creeer_incident_lot_autre_projet(self, client_conducteur, chantier_abidjan, lot_autre_chantier):
        """Vérifie le rejet HTTP 400 si le lot n'appartient pas au chantier spécifié."""
        url = "/api/v1/incidents/"
        payload = {
            "projet_id": str(chantier_abidjan.id),
            "lot_id": str(lot_autre_chantier.id),
            "titre": "Incohérence projet/lot",
            "description": "Test de violation de la règle RG-02."
        }
        response = client_conducteur.post(url, data=payload, format="json")
        assert response.status_code == status.HTTP_400_BAD_REQUEST
```

#### La Grille d'Excellence : La Définition of Done (DoD)

##### CADRAGE STRATÉGIQUE SCQA (Minto) : LA GRILLE DE CONTRÔLE AVANT LIVRAISON

##### La DoD (Definition of Done - Définition de Fini) d'un Endpoint d'Entreprise :

1. **Architecture** : Zéro requête ORM directe dans la vue ; passage exclusif par Selector & Service.
2. **Intégrité** : Le Service est sous @transaction.atomic et appelle full\_clean().
3. **Sécurité** : Double verrou actif (Rôle Global + Affectation Projet) et test d'intrusion 403 validé.
4. **Documentation** : Schéma OpenAPI Swagger documenté avec exemples via @extend\_schema.
5. **Qualité** : Couverture de tests Pytest \$lge 90%\$ avec cas nominal (201) et cas d'erreur (400, 403).

## CHAPITRE 16 : La Boussole Métacognitive : Guide de Débogage

### Diagnostic Rapide des Erreurs et Anti-Patterns Toxiques d'Entreprise

**MODÈLE MENTAL & ANALOGIE CONCRÈTE : L'ERREUR EST LE MEILLEUR PROFESSEUR DE L'INGÉNIEUR**  
**Dédramatiser l'Erreur : L'Erreur est une Information Précieuse (Dehaene & Pólya) :**  
Un développeur novice panique quand le terminal s'affiche en rouge, copie le message au hasard sur internet ou teste des modifications frénétiques sans réfléchir (Agitation Système 1).  
**Un ingénieur expert applique l'heuristique de Pólya :** L'erreur n'est pas un échec, c'est le diagnostic le plus précis et le plus bienveillant que Python vous offre. Il vous indique la ligne exacte et la règle violée.

Le Tableau de Traduction des 6 Erreurs Django les Plus Fréquentes

| Message d'Erreur Django          | Ce que cela signifie réellement                                                             | L'Action Corrective Immédiate                                              |
|----------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| OperationalError: no such column | Vous avez ajouté un champ dans models.py mais la migration n'a pas tourné.                  | Lancez makemigrations puis migrate dans le terminal.                       |
| FieldError: Unknown field...     | Faute de frappe dans le nom d'un champ dans fields du sérialiseur ou de l'admin.            | Vérifiez l'orthographe exacte dans models.py.                              |
| DoesNotExist: matching query...  | Vous avez appelé .get() sur un ID qui n'existe pas en base.                                 | Utilisez get_object_or_404() ou interceptez l'exception dans le sélecteur. |
| TypeError: ... argument 'user'   | Oubli de passer un argument obligatoire dans la signature du service.                       | Vérifiez les paramètres nommés de votre fonction de service.               |
| CSRF Failed: cookie not set      | CSRF (Cross-Site Request Forgery - Falsification de Requête Inter-Sites) rejeté en session. | Passez par l'authentification par Token JWT (Authorization: Bearer).       |

**PIÈGE DÉBUTANT & SYSTÈME 1 (Kahneman) : LES 4 ANTI-PATTERNS À BANNIR DÉFINITIVEMENT**  
**LES 4 ANTI-PATTERNS TOXIQUES D'ENTREPRISE À PROSCRIRE :**  
1. **serializer.save() dans une vue** : Interdit. L'écriture appartient au Service.  
2. **Boucle N+1** : Faire for incident in incidents: print(incident.projet.nom) sans select\_related('projet'). Cela déclenche 1000 requêtes SQL au lieu d'une seule !  
3. **Importer Request dans un Service** : Détruit la testabilité et empêche l'exécution en tâche de fond.  
4. **Suppression physique directe en SQL** : Détruit la traçabilité légale exigée chez SOUMAFE SARL.

## CHAPITRE 17 : Le Learning Wrapper & Looking Back Final

### Phase 5 : Consolidation Métacognitive, Transfert Universel & Growth Mindset

#### LOOKING BACK & MÉTACOGNITION (George Pólya) : L'HEURISTIQUE DU LOOKING BACK ET LE TRANSFERT

##### L'Analyse Rétrograde Finale (George Pólya - How to Solve It) :

Regardez le chemin parcouru depuis la page 1 : vous êtes parti de l'analogie du restaurant sans aucune notion de Django, et vous comprenez désormais comment articuler des modèles abstraits, des migrations, des sélecteurs optimisés, des services transactionnels étanches et des tests d'intrusion 403 !

**Le Transfert Universel** : Ce même patron architectural MECE (Mutually Exclusive, Collectively Exhaustive - Mutuellement Exclusif, Collectivement Exhaustif) s'applique à 100% des briques de CCD Digital :

• Gestion des Matériels & Engins de Chantier • Gestion des Pointages Ouvriers • Suivi des Bons de Commande

Le moule est invariable : `ModeleBase`  $\rightarrow$  `selectors`  $\rightarrow$  `services`  $\rightarrow$  `serializers`  $\rightarrow$  `views`  $\rightarrow$  `tests`.

#### Votre Carnet de Bord Réflexif (À Conserver Précieusement)

| Question Métacognitive                                                                            | Votre Auto-Diagnostic en Entreprise                                                                         |
|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>1. Diagnostic de Clarté</b><br>Comprenez-vous la différence entre un sélecteur et un service ? | Un sélecteur ne fait que LIRE. Un service MODIFIE la base sous transaction atomique.                        |
| <b>2. Chasse à la Régression</b><br>Avez-vous testé le cas limite d'une donnée invalide ?         | Vérifier qu'un test automatisé attend le code 400 Bad Request sur payload invalide.                         |
| <b>3. Contrôle Anti-Fuite</b><br>L'accès est-il protégé au niveau du projet ?                     | La permission <code>MembreDuProjet</code> a-t-elle été vérifiée sur l'objet ciblé ?                         |
| <b>4. Découverte des Modèles</b><br>Le modèle est-il déclaré dans <code>__init__.py</code> ?      | Sans export explicite dans <code>models/__init__.py</code> , <code>makemigrations</code> ignore le modèle ! |

#### DÉFI STIMULANT EN ZONE PROXIMALE (Ericsson) : LE MOT DU LEAD ARCHITECTE

Bravo pour avoir complété ce cursus d'ingénierie active !

Vous disposez désormais de toutes les clés conceptuelles et pratiques pour concevoir, maintenir et faire évoluer des API professionnelles irréprochables au sein de **SOUMAFE SARL**. Faites de cette rigueur votre signature technique !